



Sudoku app in NatID



Anđela Maksimović (20009), Andrijana Kešelj(20008) and Azra Kurić(20113)





Solving Algorithm



Backtracking is a systematic trial-and-error approach that:

- Tries placing numbers 1-9 in empty cells
- Validates each placement against Sudoku rules
- Continues forward if valid, backtracks if invalid
- Guarantees finding a solution if one exists



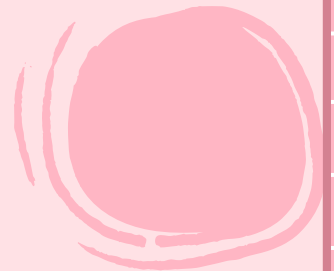


Solving Algorithm



Core Logic:

1. Find next empty cell
2. Try numbers 1-9:
 - If number is valid → place it and recurse
 - If recursion succeeds → solution found!
 - If recursion fails → remove number (backtrack)
3. If all numbers fail → return failure (backtrack further)
4. If no empty cells → puzzle solved!





Solving Algorithm



Initial puzzle (simplified):

[5, 3, _, _, 7, ...]



Empty cell

Try 1: Can it be 1? → Check row, column, box → ❌

Invalid

Try 2: Can it be 2? → Check row, column, box →

❌ Invalid

Try 4: Can it be 4? → Check row, column, box →

✅ Valid!

Place 4, move to next empty cell, repeat process...

Dead End Scenario:

- Later cells have no valid options
- Backtrack: Undo last placement
- Try next number in previous cell
- Continue until solution found





Hints Engine



Instead of just showing the answer, our hint engine teaches solving strategies that players can apply themselves.

Bad Approach : "Put 7 in this cell"

Good Approach : "This cell can only be 7 because all other numbers already exist in its row, column, or box"





Hints Engine



Five Core Techniques:

1. **Naked Singles:** Cell has only one possible valid number
 - Example: "Cell (4,5) can only be 9 - all others eliminated"
2. **Hidden Singles:** Number can only go in one place within a row/column/box
 - Example: "9 must go here - it's the only valid spot for 9 in this row"
3. **Naked Pairs:** Two cells share same two candidates, eliminating them elsewhere
 - Example: "Cells (5,2) and (5,6) must be {3,7}, remove 3 and 7 from other cells"
4. **Pointing Pairs:** Candidate restricted to one row/column within a box
 - Example: "4 in this box only appears in row 2, eliminate 4 from rest of row 2"
5. **Box/Line Reduction:** Candidate in a line restricted to one box
 - Example: "6 in row 4 only fits in box 5, eliminate 6 from rest of box 5"

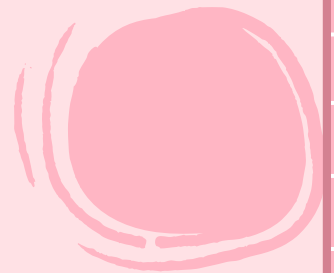


Hints Engine



User Experience Flow

1. Player clicks "Get Hint" button
2. Engine analyzes board
3. Visual highlighting appears on board
4. Text explanation shows in interface
5. Player understands and applies the technique





Board Generation Process



The generation process begins by filling the three diagonal 3×3 boxes with shuffled numbers (1-9), then uses a backtracking solver with MRV heuristic to complete the remaining cells, creating a fully solved board. Numbers are then strategically removed one at a time, with each removal validated to ensure the puzzle maintains exactly one unique solution. The system attempts up to 50 different removal patterns, keeping the best result that meets the target difficulty requirements.



Board Generation Process



Intelligent Difficulty Levels

Difficulty is determined by two factors: the number of given clues and the maximum technique level required to solve the puzzle.

Easy (40 clues): Requires only basic techniques (Level 1) - Naked Singles and Hidden Singles

Medium (32 clues): Requires intermediate techniques (Level 2+) - Naked Pairs or Pointing Pairs

Hard (26 clues): Requires advanced techniques (Level 3+) - Naked Triples or Hidden Pairs

The Hint Engine analyzes each generated puzzle by simulating the solving process, tracking which techniques are needed, ensuring players face appropriate challenges for their skill level.

